Assignment: AI-Assisted Macroeconomic Modeling Dashboard

ECO 317 - Intermediate Macroeconomic Theory

Professor Jonathan Wolff

Spring 2026

Format: Group Project (Max 6 students per group)

Tools: Python, Streamlit, Generative AI (ChatGPT/Gemini/Claude)

1. Overview

In our previous assignment, we focused on confronting macroeconomic theory with real-world evidence by building an automated data pipeline. However, data alone is not enough; we must also be able to simulate the theoretical mechanics of the economy. Over the past few weeks, we have extensively studied the household's consumption and savings decisions. Solving these dynamic models—particularly when introducing nonlinear preferences, capital production, and endogenous labor—traditionally required tedious programmatic loops.

In this assignment, you will shift from data ingestion to model simulation. Using Generative AI tools, you will act as software architects to construct an interactive, web-based Macroeconomic Dashboard using Python and the Streamlit library.

The catch: You are not expected to write every line of Python from scratch. You are expected to engineer the correct prompts, use AI to do the heavy lifting of writing the dynamic programming and simulation code, and ensure that the economic logic of the underlying Bellman and Euler equations remains perfectly sound.

2. Project Requirements

Your Streamlit application must successfully feature three distinct macroeconomic models from our course notes, integrated into a unified interactive dashboard.

Phase I: The Stochastic Models The background Python code must rigorously solve the following three models over an infinite horizon using Value Function Iteration (VFI):

- **Model 1: Stochastic Consumption-Savings Model.** A model featuring Constant Elasticity of Substitution (CES) preferences (e.g., $U(C_t) = (C_t^{1-\sigma} - 1)/(1-\sigma)$ and an

exogenously determined endowment. The code should solve for optimal policy functions subject to a two-state Markov shock to income.

- **Model 2: The Stochastic Robinson Crusoe Economy.** A model introducing production, where the consumer moves resources forward in time by accumulating productive capital (Kt+1) instead of a risk-free asset, and tomorrow's output is defined by a production function $Y_{t+1} = F(K_{t+1})$. This model must include a Markov shock to Total Factor Productivity (TFP).
- **Model 3: Endogenous Labor Supply.** A model where the household receives a time endowment ($T = 1$) and must optimally choose both consumption and labor supply (L_t), balancing the marginal utility of consumption against the marginal utility of leisure ($1 - L_t$). This model must include a Markov shock to the wage rate.

Phase II: Simulation & Exact Non-Linear Forecasting Once the policy functions are solved, your program must simulate these models.

- **Simulations:** For each model, simulate the time series of consumption, savings (or capital accumulation), and labor (where applicable) over a set time horizon (minimum 100 periods). * **Moments:** The app must also calculate and display summary statistics (mean, variance) for the simulated model moments, alongside first-order autocorrelations and correlations with aggregate output/income.
- **Forecasting:** Finally, the app must provide forecasts for key economic series (consumption, savings, labor, etc).

Phase III: The Streamlit Dashboard (Visualization) Your Python script must utilize Streamlit to generate a clean, interactive user interface.

- **Academic Styling:** The dashboard must be visually polished, featuring a dark/navy background, bright white charting boxes for contrast, and utilizing the Garamond font throughout.
- **Interactivity:** Include sidebar sliders that allow the user to easily change features of the model (e.g., the discount factor β, the risk aversion parameter σ, the real interest rate rt, or initial wealth), as well as the Markov transition matrix probabilities.
- **Visuals:** The main panel should display clean, well-labeled visualizations comparing the time series of the forecasts and mapping how the optimal consumption bundles shift when parameters change. Include a "Static Intuition" section (e.g., graphing the intra-temporal labor supply curve or mapping steady-state capital against patience).
- **AI "Intelligent" Summaries:** Incorporate dynamic text blocks beneath your charts that utilize Python f-strings to automatically generate written economic analysis based on the live mathematical moments of your simulations.

- **Navigation:** Include a way for the user to toggle smoothly between Model 1, Model 2, and Model 3.

3. AI Prompting Guide & Helpful Tips

To achieve a dashboard of this complexity, you must be highly specific when prompting ChatGPT, Gemini, or Claude.

- **Key Phrases to Include in Prompts:** * *"Solve via Value Function Iteration (VFI)."*
    - *"Discretize the state space using a fine grid (e.g., 400 points) to ensure smooth simulations."*
    - *"Use numpy.interp to evaluate continuous off-grid points during the simulation."*
    - *"Wrap the solver function in Streamlit's @st.cache_data decorator so the app does not freeze when I move a slider."*
- **Example Prompt for Forecasting:** *"Create a Streamlit layout where the user can select 'High' or 'Low' shocks for the next 3 periods from a dropdown menu. Use the exact policy functions from the VFI solver to generate a 10-period forecast of consumption and capital based on that specific shock path."*
- **Suggestions for Added Features:** Consider adding a toggle that compares two households (one with high risk aversion, one with low) on the same graph, or calculating the variance ratio of consumption to income to visually prove the Permanent Income Hypothesis.

4. The "Open Source" & AI Approach

- **AI Usage:** You are explicitly encouraged to use AI (ChatGPT, Gemini, Claude) to generate your Python and Streamlit code. However, you must understand how it works. If the AI hallucinates a mathematical function or fails to properly enforce a budget constraint, you must catch it and fix it (using AI, not your own programming).
- **Documentation:** Your code should be heavily commented. You can use AI to generate these comments to explain what each mathematical block does.

5. Deliverable: The Live Demo

There is no written report for this assignment. Instead, your grade is based entirely on a live demonstration.

- **Scheduling:** Teams will sign up for a 15-minute appointment slot with me.

- **The Demo:** You will bring your laptop, open your terminal, run streamlit run app.py from scratch in front of me, and show that it works.
    - Does the dashboard load cleanly?
    - Do the interactive sliders update the forecasting graphs in real-time?
    - Are the simulated moments mathematically accurate to the models we discussed in class?
- **Q&A:** I will ask random group members specific questions about the economic logic (e.g., "Show me the line of code where the Euler equation is solved," or "Why does increasing the discount factor (β) lower the marginal propensity to consume in this model?") and the workflow (e.g., "Explain how your prompt generated this specific Streamlit widget").